Title: Ride Booking Platform

Assessment Overview

This assessment requires you to work with a microservices-based application for a ride-booking platform. The application consists of several separate services, including ride-service, passenger-service, and notification-service. The system uses a service discovery mechanism (eureka-server) and an API gateway to manage and route requests to the appropriate microservice.

Your main task is to implement and verify the business logic for each service and ensure they can communicate with each other to perform core tasks, such as booking a ride, tracking its status, and sending notifications to users.

The overall goal is to demonstrate your ability to develop and test a distributed system.

Difficulty Level

Practitioner

Duration

120 minutes

Type

Assessment Project

What you will learn

By the end of this, you will be able to:

- Design and implement microservices using **Spring Boot** and **Spring Cloud**.
- Use **Spring Data JPA** with **Hibernate 5.x** for CRUD operations and custom queries.
- Work with **Java 8 Date/Time API** (LocalDate and LocalDateTime).
- Implement **Feign Client-based communication** between microservices.
- Configure **Eureka** for service discovery.
- Spring Cloud Load Balancer at client side for load balancing.
- Route requests via an **API Gateway**.
- Integrate a Spring Boot Ride Service with AWS S3 using AWS SDK for Java (v2).

- Upload JSON ride booking records (RideId, PassengerId, Status, Timestamp) to an S3 bucket.
- Trigger an AWS Lambda function automatically when new booking files are uploaded.
- Validate ride booking data inside Lambda and log results to CloudWatch Logs.
- Configure IAM roles and permissions for secure integration between Ride Service, S3, and Lambda.

What you will be provided

- Boilerplate code to develop the complete system.
- The boilerplate code is available in the lab **Desktop** > **Project** folder.
- Instructions to guide you through the assessment tasks.
- A Java development environment pre-configured for you to start coding immediately.
- Pre-configured AWS account access with an S3 bucket (ride-booking-records).
- Boilerplate code for AWS SDK (v2) integration in the Ride Service.
- A Lambda starter template (ProcessEnrollmentLambda) for validating ride booking records.
- Step-by-step instructions for wiring S3, Lambda, and CloudWatch together.

What you need to know

- Knowledge of Java programming.
- Understanding of **Spring Boot, and Spring Data JPA**.
- Basics of **Spring Cloud Feign Client, Eureka, Spring Cloud Gateway, Spring Cloud LoadBalancer**.
- Working with **RabbitMQ messaging**.
- How to work on **IntelliJ IDEA**.
- Basic knowledge of **AWS S3, Lambda, and IAM**.
- Familiarity with **Spring Boot** application development.
- Understanding of **JSON serialization** in Java.
- Comfort with **event-driven processing** and AWS CloudWatch logging.

Skill Tags

- Java 8
- Spring Boot
- MySQL Database
- Spring Security
- Spring Cloud Feign Client
- Spring Cloud Eureka
- Spring Cloud Gateway
- Spring Cloud LoadBalancer
- RabbitMQ

- AWS S3
- AWS Lambda

- AWS IAM
- AWS CloudWatch Logs

What you will do

You are tasked with building a **Ride Booking Platform** using Spring Boot microservices.

This system will allow:

- Users to register as passengers.
- Passengers to book rides.
- Ride Service to notify the Notification Service asynchronously (/RabbitMQ).
- Drivers and rides to be matched dynamically.
- Data stored in **MySQL databases** with appropriate entity relationships.
- Communication between services using **Feign Clients**.
- All success/failure responses to use appropriate **HTTP status codes**.
- Logging of all API requests and inter-service communications.
- **S3 Bucket Integration** – Extend the Ride Service so that whenever a ride is booked, a JSON file (containing RideId, PassengerId, Status, Timestamp) is uploaded into the ride-booking-records S3 bucket.
- **Lambda Function** – Create a Lambda function (ProcessEnrollmentLambda) that is triggered whenever a new booking file is uploaded. The function will:
- Retrieve and validate the booking JSON.
- Log processing results to **CloudWatch Logs**.
- **IAM Roles & Permissions** –
- Ride Service → s3:PutObject permission for the bucket.
- Lambda → s3:GetObject permission for the bucket.

Objectives

Ride Service

- POST /api/rides → Book a new ride.
- GET /api/rides/{id} → Retrieve ride details and status.
- Entity design:
    - **Ride Entity**
        - id: Long → Primary key, auto-generated (@Id, @GeneratedValue).
        - pickupLocation: String → Pickup location.
        - dropLocation: String → Drop-off location.
        - rideTime: LocalDateTime → Booking timestamp.
        - status: String → Ride status (BOOKED, CONFIRMED, COMPLETED, CANCELLED).
        - passengerId: Long → Foreign key (linked with Passenger Service).

- **Database**:
    - Dedicated MySQL database for Ride Service.
    - Native SQL: query rides by status count.
- **Messaging:**
    - On ride booking, publish RideBooked event to RabbitMQ.
    - Notification Service consumes this event.
- **Service discovery & communication:**
    - Register with **Eureka**.
    - Expose APIs via **API Gateway**.

Passenger Service

- POST /api/passengers → Register a new passenger.
- GET /api/passengers/{id} → Fetch passenger details.
- **Entity design:**

  Passenger Entity
    - id: Long → Primary key, auto-generated (@Id, @GeneratedValue).
    - name: String → Passenger name.
    - email: String → Unique email address.
    - Phone: String
- **Database:**
    - Dedicated MySQL database for Passenger Service.
- **Service discovery & communication:**
    - Register with **Eureka**.
    - Expose APIs via **API Gateway**.

Notification Service

- Consumes ride booking events from RabbitMQ.
- Sends confirmation (simulated via console log).

Eureka Server

- Set up Eureka Server for service discovery.
- Ride Service, Passenger Service, and Notification Service register automatically.

API Gateway

- Route all requests (Ride & Passenger endpoints) through API Gateway instead of direct URLs.
- Configure routing rules and verify forwarding to correct services.

Load Balancing

- Run multiple instances of Ride/Passenger Service.

- Configure Spring Cloud LoadBalancer with Eureka for round-robin request distribution.
- Ensure Feign Clients use Eureka service names (not hardcoded URLs).

General Requirements

- Apply LocalDate for ride scheduling.
- Implement **native SQL queries** (e.g., rides booked per passenger).

Activities

1. Important to Note

1. You are provided with a boiler plate code in the Project folder in the Lab Desktop.
2. The boilerplate code has necessary instructions in addition to what is given under the activities to develop the complete system. Please follow the instructions.
3. Do not modify the Default Code blocks given in the boiler plate code.
4. Open the Project folder in your Java IDE to develop the code.
5. Run mvn clean install from the root directory to build all modules simultaneously, or execute it manually within each specific service directory.
6. Ensure **RabbitMQ (RMQ)** are up and running before launching the applications.
7. Ensure eureka-server, api-gateway, passenger-service, ride-service, and notification-service are running simultaneously before submission.
8. Do not stop or terminate any application or background service. If these services are not running, **the automated evaluation will timeout and fail**.

2. Define the Controller Class

RideController (@RestController) mapped to /api/rides

- POST /api/rides → Book a ride.
- GET /api/rides/{id} → Retrieve ride details.

PassengerController (@RestController) mapped to /api/passengers

- POST /api/passengers → Register a new passenger.
- GET /api/passengers/{id} → Retrieve passenger details.

3. Configure Spring Cloud

- **Eureka Server** → Service registry for Ride and Passenger services.
- **API Gateway** → Route requests to Ride and Passenger services.
- **Feign Client** → Used in Ride Service to call Passenger Service.
- **Load Balancer** → Round-robin distribution when multiple instances of a service are running.

4. Configure Messaging (RabbitMQ)

- Ride Service publishes ride booking events.

- Notification Service consumes them.

6. Configure Database Properties

- Configure application.properties in each service to connect to its separate MySQL database.

8. Test all the API Endpoints using postman

Use a tool like Postman to test the following flows:

- **Ride Management:**
  - Book a ride → return 201 Created.
  - Retrieve ride details by ID.
  - Query rides by status count (native SQL).
- **Passenger Management:**
  - Register new passenger → return 201 Created.
  - Retrieve passenger details.
- **Notification:**
  - Booking a ride → triggers Rabbit MQ event → Notification Service logs confirmation.
- **API Gateway:**
  - Access Ride & Passenger endpoints via Gateway.
- **Load Balancing:**
  - Run multiple Ride/Passenger Service instances → verify round-robin distribution.
- **Logging:**
  - Verify logs for method, URL, timestamp, and status code.

9. Containerization

- Build the project JARs
  - Run mvn clean package -DskipTests to generate JAR files for all services.
- Create Dockerfiles
  - Add a Dockerfile in each service folder. Use a base image like eclipse-temurin:17-jdk, copy the built JAR, and set the startup command (java -jar app.jar).
- Build Docker images
  - Navigate to each service folder and run the following, using the recommended image names:
    - **docker build -t eureka-server ./eureka-server**
    - **docker build -t api-gateway ./api-gateway**
    - **docker build -t notification-service ./notification-service**
    - **docker build -t passenger-service ./passenger-service**
    - **docker build -t ride-service ./ride-service**
  - Note: Host ports are mapped differently in Docker Compose to avoid conflicts if the default ports are already in use on your machine. Container ports remain unchanged.
- Docker Compose configuration

- Define all services (app containers + dependencies like MySQL, RabbitMQ, etc.) with environment variables, service dependencies, and port mappings.

Service	Docker Image Name	Container Port	Host Port
Eureka Server	`eureka-server`	8761	8762
API Gateway	`api-gateway`	8089	8090
Notification Service	`notification-service`	8084	8085
Passenger Service	`passenger-service`	8083	8086
Ride Service	`ride-service`	8082	8087
MySQL Database		3306	3308
RabbitMQ		5672 / 15672	5673 / 15673

10. Cloud Integration (AWS)

a. S3 Bucket Integration

- Ride Service uploads JSON file (RideId, PassengerId, Status, Timestamp) into ride-booking-records bucket.
- Ensure AWS SDK for Java (v2) is integrated for S3 interaction.

b. Lambda Function

- Create a Lambda function (ProcessEnrollmentLambda) which is triggered whenever a new object is created in the passenger-ride-booking bucket.
- Lambda Function validates the booking record and writes a log entry (CloudWatch Logs).

c. IAM Role & Permissions

- Ride Service → s3:PutObject permissions.
- Lambda → s3:GetObject permissions.

d. Testing

- Book ride → JSON uploaded to S3.
- Lambda triggered → CloudWatch logs updated

1. Database & Persistence

Passsenger Service

Database

- Database Type: MySQL Database
- Database Name: passenger_service
- Username: root
- Password: root

- Table Name: passengers
- Ride Service Database
  - Database Type: MySQL Database
  - Database Name: ride_service
  - Username: root
  - Password: root
  - Table Name: rides

- **Project Build Status:** Ensure successful project build by executing the command mvn clean install in the directory containing the source folder and pom.xml file to generate the JAR file in each microservice.

2. Test the API Endpoints

3. Use Postman:

1. Create Passenger

URL: http://localhost:8083/api/passengers

Valid Request Body:

```
{
  "name": "John Doe1",
  "email": "john1@example.com",
  "phone": "+1-202-555-0173"
}
```

Response Body (201 Created):

```
{
    "id": 1,
    "name": "John Doe1",
```

```
    "email": "john1@example.com",
    "phone": "+1-202-555-0173"
}
```


Invalid Request Body:


```
{
  "name": "",
  "description": ""
}
```


Invalid Response (400 Bad Request):


```
{
  "timestamp": 1758158011963,
  "status": 400,
  "errors": [
    "Instructor is mandatory",
    "Name is mandatory"
  ]
}
```


2. Book ride

URL: http://localhost:8082/api/rides

Ride Valid

Request Body:


```
{
  "pickupLocation": "MG Road, Bangalore",
  "dropLocation": "Electronic City, Bangalore",
  "rideTime": "2025-09-25T10:30:00",
  "status": "BOOKED",
  "passengerId": 1
}
```


Valid Response (201 OK):


```
{
    "id": 20,
    "pickupLocation": "MG Road, Bangalore",
    "dropLocation": "Electronic City, Bangalore",
```

```
    "rideTime": "2025-10-08T17:46:08.0776617",
    "status": "BOOKED",
    "passengerId": 1
}
```


Invalid Request Body:


```
{
  "name": "",
  "email": ""
}
```


Invalid Response (400 Bad Request):


```
{
  "timestamp": 1758158550459,
  "status": 400,
  "errors":
    [ "Name is
    mandatory",
    "Email is
    mandatory"
  ]
}
```


Testcases:

Backend Testcases:

Testcase Number	Testcase Description	Marks
1	Verify Passenger Creation	11
2	Verify Get Passenger By ID	11
3	Verify Ride Booked Successfully	11
4	Verify Get Ride By ID	11
5	Verify Passenger Creation VIA Eureka	5
6	Verify Ride Booking VIA Eureka	5

7	Verify Ride Confirmation Event in Notification Service	11

AWS testcases:

Testcase Number	Testcase Description	Marks
8	Validate S3 Bucket Check	7
9	Validate Lambda Function Exists	11
10	Verify IAM Role Permissions	7

Docker testcases:

Testcase Number	Testcase Description	Marks
11	Validate eureka service image and port	2
12	Validate api gateway image and port	2
13	Verify notification service image and port	2
14	Verify ride service image and port	2
15	Verify passenger service image and port	2